

Use LiveData with ViewModel

1. Before you begin
2. Starter app overview
3. What is LiveData
4. Add LiveData to the current scrambled word
5. Attach observer to the LiveData object
6. Attach observer to score and word count
7. Use LiveData with data binding
8. Add data binding variables
9. Use binding expressions
10. Test Unscramble app with Talkback enabled
11. Delete unused code
12. Solution code
13. Summary
14. Learn more

1. Before you begin

You have learned in the previous codelabs, how to use a [ViewModel](#) to store the app data. `ViewModel` allows the app's data to survive configuration changes. In this codelab, you'll learn how to integrate [LiveData](#) with the data in the `ViewModel`.

The `LiveData` class is also part of the [Android Architecture Components](#) and is a data holder class that can be observed.

Prerequisites

- How to download source code from GitHub and open it in Android studio.
- How to create and run a basic Android app in Kotlin, using activities and fragments.
- How the activity and fragment life cycles work.
- How to retain UI data through device-configuration changes using a [ViewModel](#).
- How to write lambda expressions.

What you'll learn

- How to use [LiveData](#) and [MutableLiveData](#) in your app.
- How to encapsulate the data stored in a `ViewModel` with `LiveData`.

- How to add observer methods to observe changes in the `LiveData`.
- How to write binding expressions in a layout file.

What you'll build

- Use `LiveData` for the app's data (word, word count and the score) in the [Unscramble](#) app.
- Add observer methods that get notified when the data changes, update the scrambled word text view automatically.
- Write binding expressions in the layout file, which are triggered when the underlying `LiveData` is changed. The score, word count and the scrambled word text views are updated automatically.

What you need

- A computer with Android Studio installed.
- Solution code from the previous codelab (Unscramble app with `ViewModel`).

Download the starter code for this codelab

This codelab uses the Unscramble app that you built in the previous codelab ([Store data in ViewModel](#)) as the starter code.

Starter Code URL: <https://github.com/google-developer-training/android-basics-kotlin-unscramble-app/tree/starter>

Add the solution code from the previous codelab ([Store data in ViewModel](#)) to the above starter branch, and use it as starter code for this codelab.

2. Starter app overview

This codelab uses the Unscramble solution code that you are familiar with from the previous codelab. The app displays a scrambled word for the player to unscramble it. The player can try any number of times to guess the correct word. The app data such as the current word, player's score and word count are saved in the `ViewModel`. However, the app's UI does not reflect the new score and word count values. In this codelab, you will implement the missing features using `LiveData`.

11:48



Unscramble

7 of 10 words

SCORE: 60

anoblol

Unscramble the word using all the letters.

Enter your word

SKIP

SUBMIT



3. What is LiveData

[LiveData](#) is an observable data holder class that is lifecycle-aware.

Some characteristics of `LiveData`:

- `LiveData` holds data; `LiveData` is a wrapper that can be used with any type of data.
- `LiveData` is observable, which means that an observer is notified when the data held by the `LiveData` object changes.
- `LiveData` is lifecycle-aware. When you attach an observer to the `LiveData`, the observer is associated with a [LifecycleOwner](#) (usually an activity or fragment). The `LiveData` only updates observers that are in an active lifecycle state such as [STARTED](#) or [RESUMED](#). You can read more about `LiveData` and observation [here](#).

UI updation in the starter code

In the starter code the `updateNextWordOnScreen()` method is called explicitly, every time you want to display a new scrambled word in the UI. You call this method during game initialization, and when players press the **Submit** or **Skip** button. This method is called from the methods `onViewCreated()`, `restartGame()`, `onSkipWord()`, and `onSubmitWord()`. With `LiveData`, you will not have to call this method from multiple places to update the UI. You will do it only once in the observer.

4. Add LiveData to the current scrambled word

In this task, you will learn how to wrap any data with `LiveData`, by converting the current word in the `GameViewModel` to [LiveData](#). In a later task, you will add an observer to these `LiveData` objects and learn how to observe the `LiveData`.

MutableLiveData

`MutableLiveData` is the mutable version of the `LiveData`, that is, the value of the data stored within it can be changed.

1. In `GameViewModel`, change the type of the variable `_currentScrambledWord` to [MutableLiveData](#)<String>. `LiveData` and `MutableLiveData` are generic classes, so you need to specify the type of data that they hold.
2. Change the variable type of `_currentScrambledWord` to `val` because the value of the `LiveData`/`MutableLiveData` object will remain the same, and only the data stored within the object will change.

```
private val _currentScrambledWord = MutableLiveData<String>()
```

3. Change the backing field, `currentScrambledWord` type to `LiveData<String>`, because it is immutable. Android Studio will show some errors which you will fix in the next steps.

```
val currentScrambledWord: LiveData<String>
    get() = _currentScrambledWord
```

4. To access the data within a `LiveData` object, use the `value` property. In `GameViewModel` inside the `getNextWord()` method, within the `else` block, change the reference of `_currentScrambledWord` to `_currentScrambledWord.value`.

```
private fun getNextWord() {
    ...
} else {
    _currentScrambledWord.value = String(tempWord)
    ...
}
```

5. Attach observer to the LiveData object

In this task you set up an observer in the app component, `GameFragment`. The observer you will add observes the changes to the app's data `currentScrambledWord`. `LiveData` is lifecycle-aware, meaning it only updates observers that are in an active lifecycle state. So the observer in the `GameFragment` will only be notified when the `GameFragment` is in `STARTED` or `RESUMED` states.

1. In `GameFragment`, delete the method `updateNextWordOnScreen()` and all the calls to it. You do not require this method, as you will be attaching an observer to the `LiveData`.
2. In `onSubmitWord()`, modify the empty `if-else` block as follows. The complete method should look like this.

```
private fun onSubmitWord() {
    val playerWord = binding.textInputEditText.text.toString()

    if (viewModel.isUserWordCorrect(playerWord)) {
        setErrorTextField(false)
        if (!viewModel.nextWord()) {
            showFinalScoreDialog()
        }
    } else {
        setErrorTextField(true)
    }
}
```

3. Attach an observer for `currentScrambledWord` `LiveData`. In `GameFragment` at the end of the callback `onViewCreated()`, call the [observe\(\)](#) method on `currentScrambledWord`.

```
// Observe the currentScrambledWord LiveData.  
viewModel.currentScrambledWord.observe()
```

Android Studio will display an error about missing parameters. You will fix the error in the next step.

4. Pass `viewLifecycleOwner` as the first parameter to the `observe()` method. The `viewLifecycleOwner` represents the [Fragment's View](#) lifecycle. This parameter helps the LiveData to be aware of the `GameFragment` lifecycle and notify the observer only when the `GameFragment` is in active states (`STARTED` or `RESUMED`).
5. Add a lambda as a second parameter with `newWord` as a function parameter. The `newWord` will contain the new scrambled word value.

```
// Observe the scrambledCharArray LiveData, passing in the LifecycleOwner and  
the observer.  
viewModel.currentScrambledWord.observe(viewLifecycleOwner,  
    { newWord ->  
    })
```

A lambda expression is an anonymous function that isn't declared, but is passed immediately as an expression. A lambda expression is always surrounded by curly braces `{ }`.

6. In the function body of the lambda expression, assign `newWord` to the scrambled word text view.

```
// Observe the scrambledCharArray LiveData, passing in the LifecycleOwner and  
the observer.  
viewModel.currentScrambledWord.observe(viewLifecycleOwner,  
    { newWord ->  
        binding.textViewUnscrambledWord.text = newWord  
    })
```

7. Compile and run app. Your game app should work exactly as before, but now the scrambled word text view is automatically updated in the LiveData observer, not in the `updateNextWordOnScreen()` method.

6. Attach observer to score and word count

As in the previous task, in this task you will add LiveData to the other data in the app, score and word count, so that the UI is updated with correct values of the score and word count during the game.

Step 1: Wrap score and wordcount with LiveData

1. In `GameViewModel`, change the type of the `_score` and `_currentWordCount` class variables to `val`.

2. Change the data type of the variables `_score` and `_currentWordCount` to `MutableLiveData` and initialize them to 0.
3. Change backing fields type to `LivData<Int>`.

```
private val _score = MutableLiveData(0)
val score: LiveData<Int>
    get() = _score

private val _currentWordCount = MutableLiveData(0)
val currentWordCount: LiveData<Int>
    get() = _currentWordCount
```

4. In `GameViewModel` at the beginning of the `reinitializeData()` method, change the reference of `_score` and `_currentWordCount` to `_score.value` and `_currentWordCount.value` respectively.

```
fun reinitializeData() {
    _score.value = 0
    _currentWordCount.value = 0
    wordsList.clear()
    getNextWord()
}
```

5. In the `GameViewModel`, inside the `nextWord()` method, change the reference of `_currentWordCount` to `_currentWordCount.value!!`.

```
fun nextWord(): Boolean {
    return if (_currentWordCount.value!! < MAX_NO_OF_WORDS) {
        getNextWord()
        true
    } else false
}
```

6. In `GameViewModel`, inside the `increaseScore()` and `getNextWord()` methods, change the reference of `_score` and `_currentWordCount` to `_score.value` and `_currentWordCount.value` respectively. Android Studio will show you an error because `_score` is no longer an integer, it's `LivData`, you will fix it in the next steps.
7. Use the [plus\(\)](#) Kotlin function to increase the `_score` value, which performs the addition with null-safety.

```
private fun increaseScore() {
    _score.value = (_score.value)?.plus(SCORE_INCREASE)
}
```

8. Similarly use [inc\(\)](#) Kotlin function to increment the value by one with null-safety.

```
private fun getNextWord() {
    ...
} else {
    _currentScrambledWord.value = String(tempWord)
    _currentWordCount.value = (_currentWordCount.value)?.inc()
```

```

        wordsList.add(currentWord)
    }
}

```

9. In `GameFragment`, access the value of `score` using the `value` property. Inside the `showFinalScoreDialog()` method, change `viewModel.score` to `viewModel.score.value`.

```

private fun showFinalScoreDialog() {
    MaterialAlertDialogBuilder(requireContext())
        .setTitle(getString(R.string.congratulations))
        .setMessage(getString(R.string.you_scored, viewModel.score.value))
        ...
        .show()
}

```

Step 2: Attach observers to score and word count

In the app, the score and the word count are not updated. You will update them in this task using `LiveData` observers.

1. In `GameFragment` inside the `onViewCreated()` method, delete the code that updates the score and word count text views.

Remove:

```

binding.score.text = getString(R.string.score, 0)
binding.wordCount.text = getString(R.string.word_count, 0, MAX_NO_OF_WORDS)

```

2. In the `GameFragment` at the end of `onViewCreated()` method, attach observer for `score`. Pass in the `viewLifecycleOwner` as the first parameter to the observer and a lambda expression for the second parameter. Inside the lambda expression, pass the new score as a parameter and inside the function body, set the new score to the text view.

```

viewModel.score.observe(viewLifecycleOwner,
    { newScore ->
        binding.score.text = getString(R.string.score, newScore)
    })

```

3. At the end of the `onViewCreated()` method, attach an observer for the `currentWordCount` `LiveData`. Pass in the `viewLifecycleOwner` as the first parameter to the observer and a lambda expression for the second parameter. Inside the lambda expression, pass the new word count as a parameter and in the function body, set the new word count along with the `MAX_NO_OF_WORDS` to the text view.

```

viewModel.currentWordCount.observe(viewLifecycleOwner,
    { newWordCount ->
        binding.wordCount.text =

```

```
        getString(R.string.word_count, newWordCount, MAX_NO_OF_WORDS)  
    })
```

The new observers will be triggered when the value of score and word count change inside the `ViewModel`, during the lifetime of the lifecycle owner, that is, the `GameFragment`.

4. Run your app to see the magic. Play the game through some words. Score and word count are also updated correctly on the screen. Observe that you are not updating these text views based on some conditions in the code. The `score` and `currentWordCount` are `LiveData` and the corresponding observers are automatically called when the underlying value changes.

12:51



Unscramble

6 of 10 words

SCORE: 60

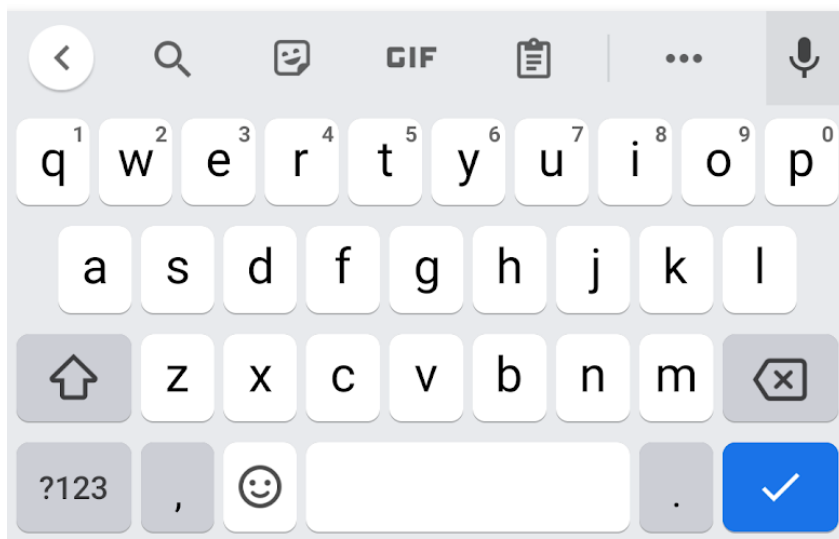
lanoujr

Unscramble the word using all the letters.

Enter your word

SKIP

SUBMIT



7. Use LiveData with data binding

In the previous tasks, your app listens to the data changes in the code. Similarly, apps can listen to the data changes from the layout. With Data Binding, when an observable `LiveData` value changes, the UI elements in the layout it's bound to are also notified, and the UI can be updated from within the layout.

Concept: Data binding

In the previous codelabs you have seen [View Binding](#), which is a one-way binding. You can bind views to code but not vice versa.

Refresher for View binding:

View binding is a feature that allows you to more easily access views in code. It generates a binding class for each XML layout file. An instance of a binding class contains direct references to all views that have an ID in the corresponding layout. For example, the Unscramble app currently uses view binding, so the views can be referenced in the code using the generated binding class.

Example:

```
binding.textViewUnscrambledWord.text = newWord
binding.score.text = getString(R.string.score, newScore)
binding.wordCount.text =
    getString(R.string.word_count, newWordCount,
MAX_NO_OF_WORDS)
```

Using view binding you can't reference the app data in the views (layout files). This can be accomplished using [Data binding](#).

Data Binding

Data Binding Library is also a part of the [Android Jetpack library](#). Data binding binds the UI components in your layouts to data sources in your app using a declarative format, which you will learn later in the codelab.

In simpler terms Data binding is binding data (from code) to views + view binding (binding views to code):

Example using view binding in UI controller

```
binding.textViewUnscrambledWord.text = viewModel.currentScrambledWord
```

Example using data binding in layout file

```
android:text="@{gameViewModel.currentScrambledWord}"
```

The above example shows how to use the Data Binding Library to assign app data to the views/widget directly in the layout file. Note the use of `@{ }` syntax in the assignment expression.

The main advantage of using data binding is, it lets you remove many UI framework calls in your activities, making them simpler and easier to maintain. This can also improve your app's performance and help prevent memory leaks and null pointer exceptions.

Step 1: Change view binding to data binding

1. In the `build.gradle (Module)` file, enable the `dataBinding` property under the `buildFeatures` section.

Replace

```
buildFeatures {  
    viewBinding = true  
}
```

with

```
buildFeatures {  
    dataBinding = true  
}
```

Do a gradle sync when prompted by Android Studio.

2. To use data binding in any Kotlin project, you should apply the `kotlin-kapt` plugin. This step is already done for you in the `build.gradle (Module)` file.

```
plugins {  
    id 'com.android.application'  
    id 'kotlin-android'  
    id 'kotlin-kapt'  
}
```

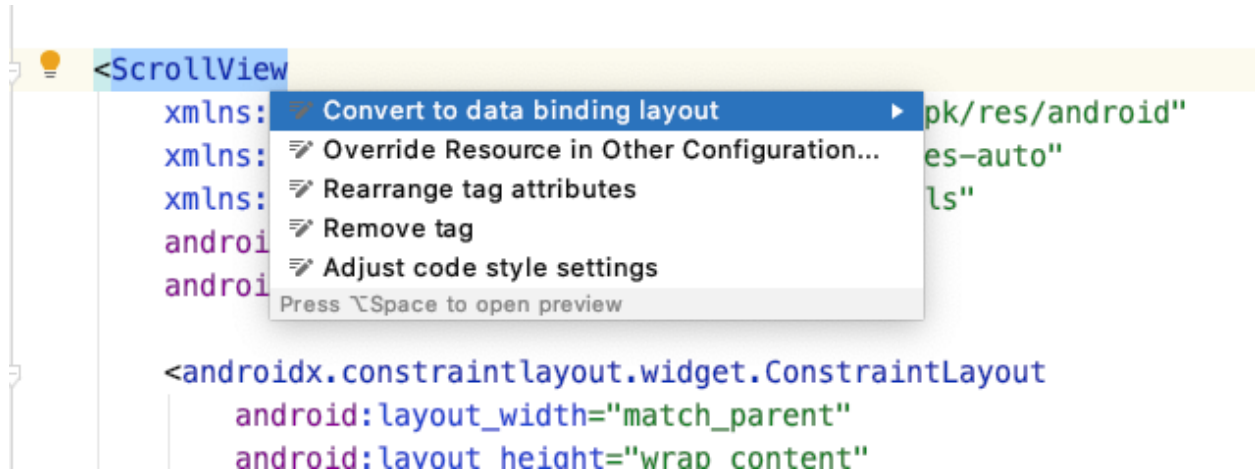
Above steps auto generates a binding class for every layout XML file in the app. If the layout file name is `activity_main.xml` then your autogen class will be called `ActivityMainBinding`.

Step 2: Convert layout file to data binding layout

Data binding layout files are slightly different and start with a root tag of `<layout>` followed by an optional `<data>` element and a `view` root element. This view element is what your root would be in a non-binding layout file.

1. Open `game_fragment.xml`, select **code** tab.

2. To convert the layout to a Data Binding layout, wrap the root element in a `<layout>` tag. You'll also have to move the namespace definitions (the attributes that start with `xmlns:`) to the new root element. Add `<data></data>` tags inside `<layout>` tag above the root element. Android Studio offers a handy way to do this automatically: Right-click the root element (`ScrollView`), select **Show Context Actions** > **Convert to data binding layout**.



3. Your layout should look something like this:

```
<layout xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools">

  <data>

</data>

  <ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.constraintlayout.widget.ConstraintLayout
      ...
    </androidx.constraintlayout.widget.ConstraintLayout>
  </ScrollView>
</layout>
```

4. In `GameFragment`, at the beginning of the `onCreateView()` method, change the instantiation of the binding variable to use data binding.

Replace

```
binding = GameFragmentBinding.inflate(inflater, container, false)
```

with

```
binding = DataBindingUtil.inflate(inflater, R.layout.game_fragment,
    container, false)
```

5. Compile the code; you should be able to compile without any issues. Your app now uses data binding and the views in the layout can access the app data.

8. Add data binding variables

In this task you will add properties in the layout file to access the app data from the `viewModel`. You will initialize the layout variables in the code.

1. In `game_fragment.xml`, inside the `<data>` tag add a child tag called `<variable>`, declare a property called `gameViewModel` and of the type `GameViewModel`. You will use this to bind the data in `ViewModel` to the layout.

```
<data>
    <variable
        name="gameViewModel"
        type="com.example.android.unscramble.ui.game.GameViewModel" />
</data>
```

Notice the type of `gameViewModel` contains the package name. Make sure this package name matches with the package name in your app.

2. Below the `gameViewModel` declaration, add another variable inside the `<data>` tag of type `Integer`, and name it `maxNoOfWords`. You will use this to bind to the variable in `ViewModel` to store the number of words per game.

```
<data>
    ...
    <variable
        name="maxNoOfWords"
        type="int" />
</data>
```

3. In `GameFragment` at the beginning of the `onViewCreated()` method, initialize the layout variables `gameViewModel` and `maxNoOfWords`.

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    binding.gameViewModel = viewModel

    binding.maxNoOfWords = MAX_NO_OF_WORDS
    ...
}
```

4. The `LiveData` is lifecycle-aware observable, so you have to pass the lifecycle owner to the layout. In the `GameFragment`, inside the `onViewCreated()` method, below the initialization of the binding variables, add the following code.

```
// Specify the fragment view as the lifecycle owner of the binding.  
// This is used so that the binding can observe LiveData updates  
binding.lifecycleOwner = viewLifecycleOwner
```

Recall that you implemented a similar functionality when implementing `LiveData` observers. You passed `viewLifecycleOwner` as one of the parameters to the `LiveData` observers.

9. Use binding expressions

Binding expressions are written within the layout in the attribute properties (such as `android:text`) referencing the layout properties. Layout properties are declared at the top of the data binding layout file, via the `<variable>` tag. When any of the dependent variables change, the 'DB Library' will run your binding expressions (and thus updates the views). This change-detection is a great optimization which you get for free, when you use a Data Binding Library.

Syntax for binding expressions

Binding expressions start with an `@` symbol and are wrapped inside curly braces `{}`. In the following example, the `TextView` text is set to the `firstName` property of the `user` variable:

Example:

```
<TextView android:layout_width="wrap_content"  
          android:layout_height="wrap_content"  
          android:text="@{user.firstName}" />
```

Step 1: Add binding expression to the current word

In this step, you bind the current word text view to the `LiveData` object in the `ViewModel`.

1. In `game_fragment.xml`, add a text attribute to the `textView_unscrambled_word` text view. Use the new layout variable, `gameViewModel` and assign `@{gameViewModel.currentScrambledWord}` to the text attribute.

```
<TextView  
    android:id="@+id/textView_unscrambled_word"  
    ...  
    android:text="@{gameViewModel.currentScrambledWord}"  
    .../>
```

2. In `GameFragment`, remove the `LiveData` observer code for `currentScrambledWord`: You don't need the observer code in fragment any more. The layout receives the updates of the changes to the `LiveData` directly.

Remove:

```
viewModel.currentScrambledWord.observe(viewLifecycleOwner,
    { newWord ->
        binding.textViewUnscrambledWord.text = newWord
    })
```

3. Run your app, your app should work as before. But now the scrambled word text view uses the binding expressions to update the UI, not the `LiveData` observers.

Step 2: Add binding expression to the score and the word count

Resources in data binding expressions

A data binding expression can reference app resources with the following syntax.

Example:

```
android:padding="@{@dimen/largePadding}"
```

In the above example, the `padding` attribute is assigned a value of `largePadding` from the `dimen.xml` resource file.

You can also pass layout properties as resource parameters.

Example:

```
android:text="@{@string/example_resource(user.lastName)}"
```

```
strings.xml
```

```
<string name="example_resource">Last Name: %s</string>
```

In the above example, `example_resource` is a string resource with `%s` placeholder. You are passing `user.lastName` as a resource parameter in the binding expression, where `user` is a layout variable.

In this step you will add binding expressions to the score and word count text views, passing in the resource parameters. This step is similar to what you did for `textView_unscrambled_word` above.

1. In `game_fragment.xml`, update the `text` attribute for `word_count` text view with the following binding expression. Use `word_count` string resource and pass in `gameViewModel.currentWordCount`, and `maxNoOfWords` as resource parameters.

```

<TextView
    android:id="@+id/word_count"
    ...
    android:text="@{@string/word_count(gameViewModel.currentWordCount,
maxNoOfWords)}"
.../>

```

2. Update the `text` attribute for `score` text view with the following binding expression. Use `score` string resource and pass in `gameViewModel.score` as a resource parameter.

```

<TextView
    android:id="@+id/score"
    ...
    android:text="@{@string/score(gameViewModel.score)}"
... />

```

3. Remove `LiveData` observers from the `GameFragment`. You don't need them any longer, binding expressions update the UI when the corresponding `LiveData` changes.

Remove:

```

viewModel.score.observe(viewLifecycleOwner,
    { newScore ->
        binding.score.text = getString(R.string.score, newScore)
    })

viewModel.currentWordCount.observe(viewLifecycleOwner,
    { newWordCount ->
        binding.wordCount.text =
            getString(R.string.word_count, newWordCount, MAX_NO_OF_WORDS)
    })

```

4. Run your app and play through some words. Now your code uses `LiveData` and binding expressions to update the UI.

3:21



Unscramble

1 of 10 words

SCORE: 0

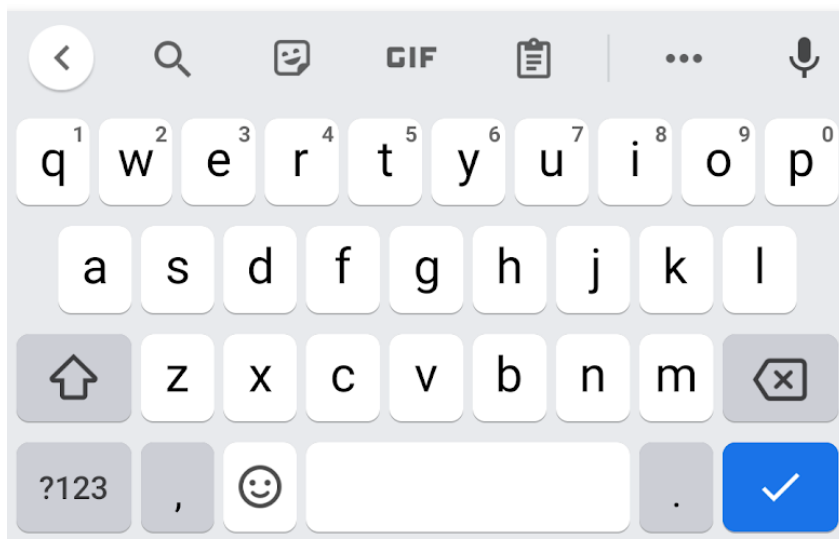
rocmosasl

Unscramble the word using all the letters.

Enter your word

SKIP

SUBMIT



3:21



Unscramble

10 of 10 words

SCORE: 120

uoy

Unscramble the word using all the letters.

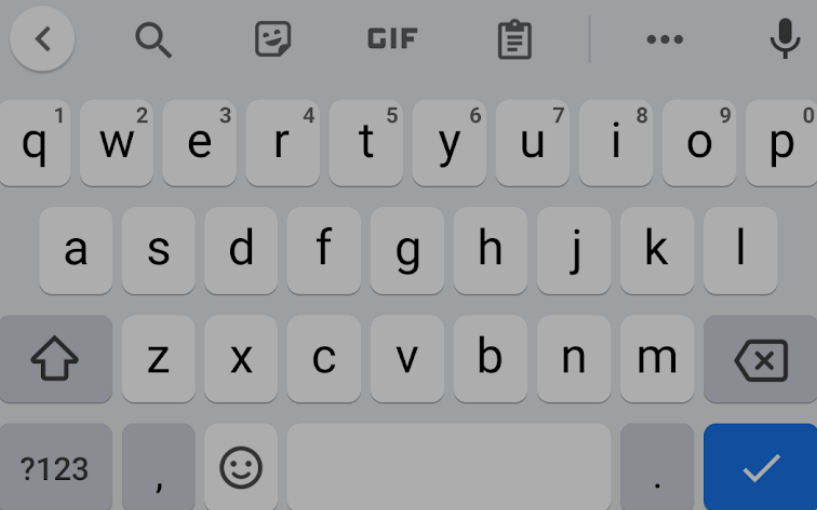
Enter your word

Congratulations!

You scored: 120

EXIT

PLAY AGAIN



Congratulations! You have learned how to use `LiveData` with `LiveData` observers and `LiveData` with binding expressions.

10. Test Unscramble app with Talkback enabled

As you've been learning throughout this course, you want to build apps that are accessible to as many users as possible. Some users may use [Talkback](#) to access and navigate your app. TalkBack is the Google screen reader included on Android devices. TalkBack gives you spoken feedback so that you can use your device without looking at the screen.

With Talkback enabled, ensure that a player can play the game.

1. Enable Talkback on your device by following these [instructions](#).
2. Return to the Unscramble app.
3. Explore your app with Talkback using these [instructions](#). Swipe right to navigate through screen elements in sequence, and swipe left to go in the opposite direction. Double-tap anywhere to select. Verify that you can reach all elements of your app with swipe gestures.
4. Ensure that a Talkback user is able to navigate to each item on the screen.
5. Observe that Talkback tries to read the scrambled word as a word. This may be confusing to the player since this is not a real word.
6. A better user experience would be to have Talkback read aloud the individual characters of the scrambled word. Within the `GameViewModel`, convert the scrambled word `String` to a `Spannable` string. A spannable string is a string with some extra information attached to it. In this case, we want to associate the string with a [TtsSpan](#) of [TYPE VERBATIM](#), so that the text-to-speech engine reads aloud the scrambled word verbatim, character by character.
7. In `GameViewModel`, use the following code to modify how the `currentScrambledWord` variable is declared:

```
val currentScrambledWord: LiveData<Spannable> =
    Transformations.map(_currentScrambledWord) {
        if (it == null) {
            SpannableString("")
        } else {
            val scrambledWord = it.toString()
            val spannable: Spannable = SpannableString(scrambledWord)
            spannable.setSpan(
                TtsSpan.VerbatimBuilder(scrambledWord).build(),
                0,
                scrambledWord.length,
                Spannable.SPAN_INCLUSIVE_INCLUSIVE
            )
            spannable
        }
    }
```

This variable is now a `LiveData<Spannable>` instead of `LiveData<String>`. You don't have to worry about understanding all the details of how this works, but the implementation uses a

`LiveData` transformation to convert the current scrambled word `String` into a `Spannable` string that can be handled appropriately by the accessibility service. In the next codelab, you will learn more about `LiveData` transformations, which allow you to return a different `LiveData` instance based on the value of corresponding `LiveData`.

8. Run the Unscramble app, explore your app with Talkback. TalkBack should read out the individual characters of the scrambled word now.

For more information on how to make your app more accessible, check out these [principles](#).

Note: The accessibility service included on your device may vary on your device depending on the device manufacturer, you may experience different behavior. If the individual characters are not being read aloud for the scrambled word, try running your app on the emulator in Android Studio. You will need to install the [Android Accessibility Suite app](#) on the emulator in order to enable Talkback.

11. Delete unused code

It is a good practice to delete the dead, unused, unwanted code for the solution code. This makes the code easy to maintain, which also makes it easier for new teammates to understand the code better.

1. In `GameFragment`, delete `getNextScrambledWord()` and `onDetach()` methods.
2. In `GameViewModel` delete `onCleared()` method.
3. Delete any unused imports, at the top of the source files. They will be greyed out.

You don't need the log statements any more, you can delete them from the code if you prefer.

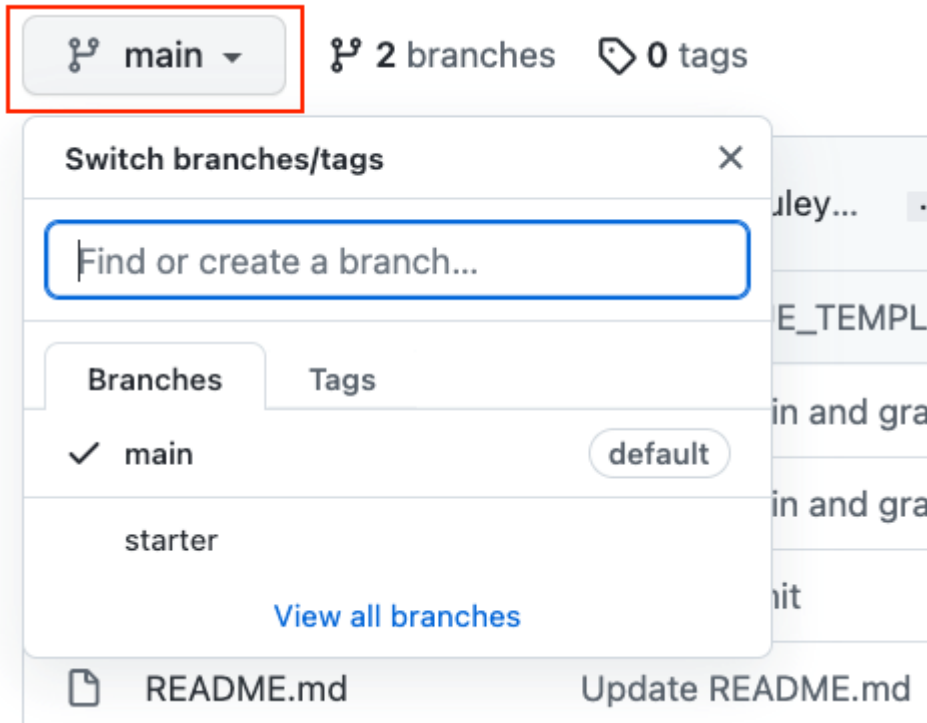
1. [Optional] Delete the `Log` statements in the source files(`GameFragment.kt` and `GameViewModel.kt`) you added in the previous codelab, to understand the `ViewModel` lifecycle.

12. Solution code

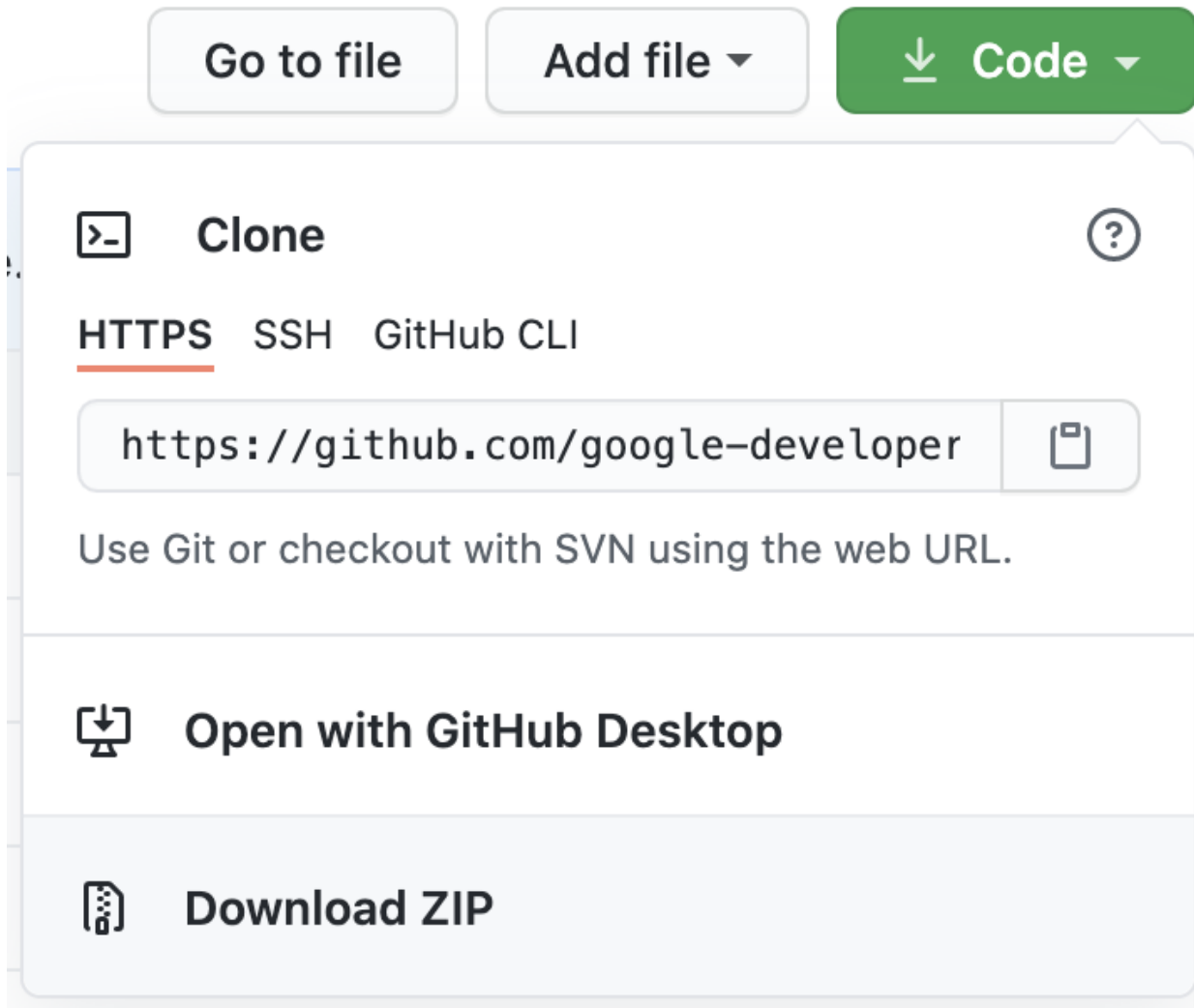
The solution code for this codelab is in the project shown below.

Solution Code URL: <https://github.com/google-developer-training/android-basics-kotlin-unscramble-app/tree/main>

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



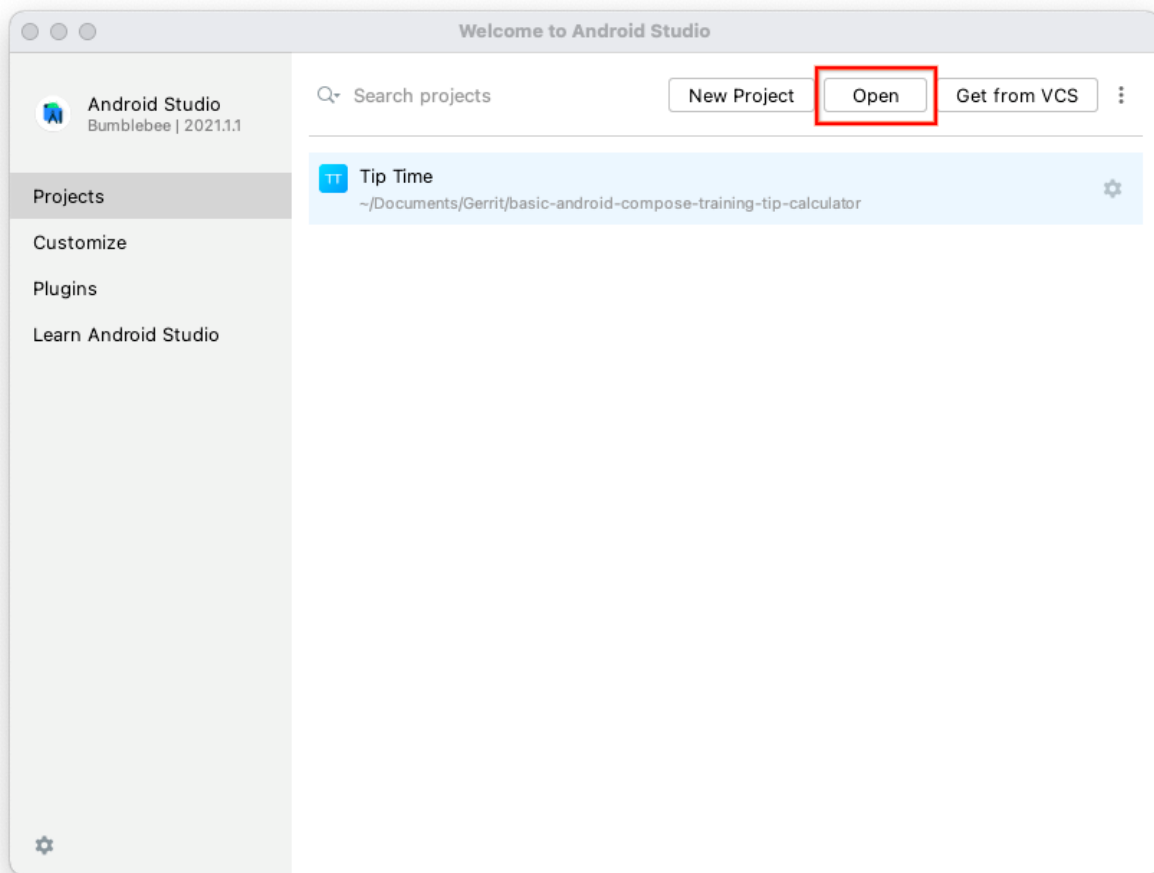
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



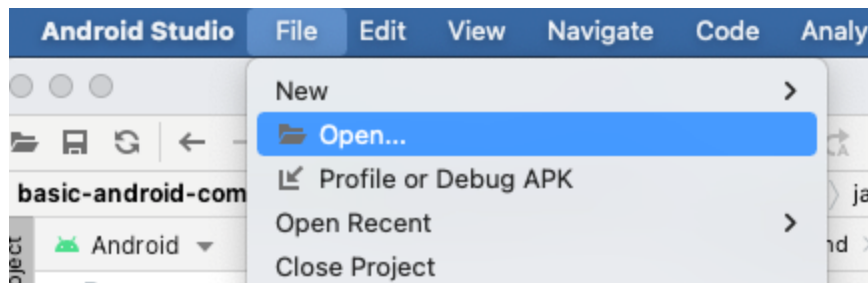
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

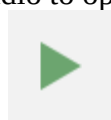
1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

13. Summary

- `LiveData` holds data; `LiveData` is a wrapper that can be used with any data
- `LiveData` is observable, which means that an observer is notified when the data held by the `LiveData` object changes.
- `LiveData` is lifecycle-aware. When you attach an observer to the `LiveData`, the observer is associated with a [LifecycleOwner](#) (usually an Activity or Fragment). The `LiveData` only updates observers that are in an active lifecycle state such as [STARTED](#) or [RESUMED](#). You can read more about `LiveData` and observation [here](#).
- Apps can listen to the `LiveData` changes from the layout using Data Binding and binding expressions.
- Binding expressions are written within the layout in the attribute properties (such as `android:text`) referencing the layout properties.

14. Learn more

- [LiveData Overview](#)
- [LiveData observer](#) API reference
- [Data binding](#)
- [Two-way data binding](#)

Blog posts

- [Data Binding — lessons learnt. The Data Binding Library \(referred to... | by Chris Banes | Android Developers](#)